



UNIVERSIDADE DE COIMBRA

FCTUC – Licenciatura em Engenharia e Ciência de Dados

Tópicos de Inteligência Artificial

2022/2023

David João Pinto Ferreira nº2020248052

João Tomás Cardoso Fernandes nº2021219036

Diogo Ramos nº2021231020

Docente responsável:
Luís Miguel Machado Lopes Macedo

Índice

Introdução	3
Desenvolvimento	4
Descrição do Agente.....	4
Descrição do Ambiente	4
Bugs e outros problemas encontrados.....	5
Testes & Resultados.....	6
Conclusão.....	8
Referencias.....	Erro! Marcador não definido.

Introdução

A área de “*Pathfinding algorithms*” tem vindo a ser grandiosamente desenvolvida nos recentes anos. Com os carros autónomos a tornarem-se cada vez mais uma realidade, maiores foram os esforços direcionados para essa área. Este assunto aprofunda algoritmos de “*Graph Search*”. Em termos mais simples o programa explora todos os caminhos existentes entre nós, encontrando assim o caminho mais eficaz tendo em consideração vários termos tais como o tempo e o custo.

“*Multi-Agent pathfinding*” é tudo o que fornecemos à “inteligência” do carro. Mistura vários agentes de “*pathfinding*” sem que estes colidam. A Teoria de Grafos (*Graph Theory*) fundamenta este trabalho.

No nosso caso estamos a trabalhar com uma espécie de um jogo de “*snake*” que contém obstáculos (paredes) e também chaves espalhadas por um mapa de jogo matricial. As paredes e as chaves são geradas de uma forma aleatório, e cada mapa difere de jogo para jogo, tornando o ambiente completamente “*random*”. O nosso problema de otimização é conseguir com que o nosso agente consiga ser capaz de aprender informação suficiente para conseguir escolher o caminho mais curto de forma a captar todas as chaves.

O que esperamos com este projeto é explorar esta área e tentar aprender a forma mais rentável de o nosso agente completar o seu objetivo sem falhas. Para isso, atribuímos valores a cada nó da nossa matriz, jogando com os valores de cada, o nosso agente será capaz de procurar independentemente qual o caminho mais eficiente de obter as chaves. Em alguns casos poderemos obter um loop infinito devido a valores iguais. Digamos que o nosso agente se encontra entre vários nós com o mesmo valor, este poderá não ser capaz de decidir nenhum nó para se mover.

A exploração desta área é bastante importante em prol de desenvolver o setor da automação. Este trabalho constante de várias partes é o que nos permitirá, como sociedade, ter um futuro maioritariamente automatizado, por exemplo, nos carros. Iremos trabalhar em cima de um projeto anterior e iremos avaliar a eficiência e comportamento do nosso agente.

Desenvolvimento

Descrição do Agente

Para o agente funcionar corretamente, utilizamos *policy iteration* para a criação de um *valueMap*, possuindo o tal sistema de *rewards*, valores estes, associados a cada posição do Maze, a função *actionRewardFunction* dá-nos a *reward* associada à próxima posição do agente para uso em outras funções no programa, o *valueMap* é também inicializado junto com os *states* possíveis e os *deltas*.

A função *LegalMoves* é fundamental no funcionamento do programa, sendo que é esta que calcula todos os movimentos possíveis que o agente pode fazer, não tendo esta em conta as *rewards*, apenas os movimentos possíveis, esse trabalho é feito pela função *ValueCheck*, que ao obter os movimentos possíveis vindos da função anterior calcula o melhor movimento possível que o agente pode tomar, tendo em conta agora as *rewards*, utilizando o *valueMap*. Finalmente esta dá *call* à função *mover* que apenas executa o movimento, atualizando a posição do agente que guardamos na variável *XYreal*.

De forma ao *valueMap* funcionar, é também fundamental apresentar que alguns parâmetros tiveram de ser selecionados à priori, principalmente *gamma*, o *discount rate* e também o número de iterações efetuadas em cada movimento, estes valores poderiam todos ser alterados, mas ao longo da execução do projeto, os utilizados enquadram-se totalmente no contexto do nosso problema.

Descrição do Ambiente

Neste projeto para a visualização do ambiente utilizamos o *PySimpleGUI*, para criar uma *simple graphical user interface*, pela razão de que a partir da pesquisa previamente efetuada em relação ao tema concordamos que o *PySimpleGUI* seria a melhor escolha.

Criamos inicialmente um menu de entrada com opções de dificuldade. Isto irá definir a criação da interface gráfica. As 3 opções – fácil, médio e difícil – irão atribuir os valores estipulados em relação às funções *nKeys*, *falhas* e *rowColCount*.

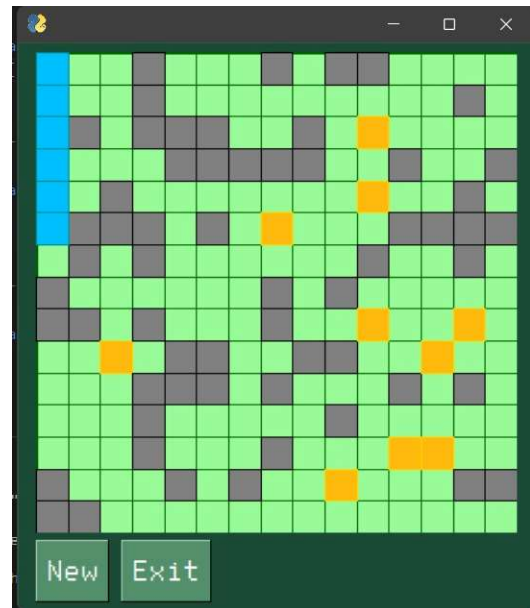
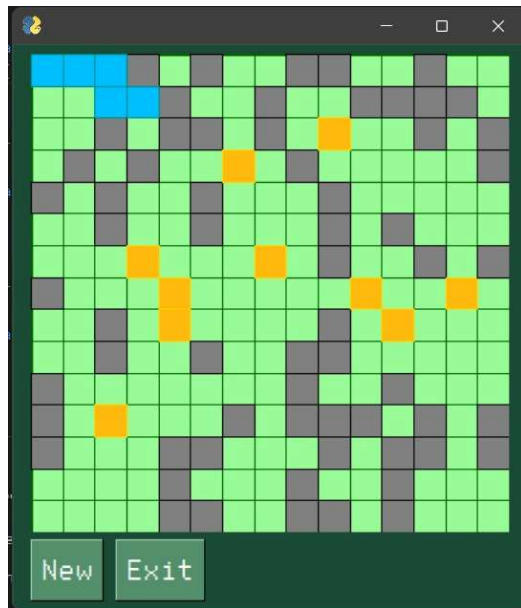
Para criar a interface gráfica temos de criar um *maze*, espaço onde o agente se irá movimentar devido à função *makeMaze* contida no código. Há a criação de um *array* de zeros, onde de seguida os valores serão alterados para 3, indicando desta forma que nessa posição do *array* está presente uma chave. Relacionado com o *maze* (denominado de *starterMap*), existe uma função denominada de *makeWeights*, em suma, faz com que as paredes indicadas, expostas através de blocos azuis, assumam o valor 1 no *starterMap*. Com o propósito de a janela da interface não sofrer alterações independentemente do seu tamanho, existe o *cellsize* para a formatação do comprimento de cada célula/quadrado onde o agente se pode movimentar, ou seja, um movimento no *starterMap* tem um tamanho diferente na janela da interface. Em relação à interface a sua construção é realizada utilizando as funções *drawGrid*, *drawCell* e *placeCells*. A função *drawGrid* cria o mapa na interface onde o agente se pode movimentar, a criação de cada célula/quadrado acontece a partir da função *drawCell* o que faz com que o agente se possa mover por células, por último para facilitar a distinção de cada célula e o que cada representa, alteramos as suas cores a partir do *placeCells* onde a cor verde representa o local onde o agente pode andar, cinzento as paredes que o agente não pode transpor, em dourado apresentam-se as chaves, e por último a cor azul indica o agente e todas as suas posições anteriores, indicando-nos assim o caminho mais eficiente. Existe um *ValueMap* que é obtido através do *reward* já indicado anteriormente, que nos indica uma matriz de posição utilizando valores, esta possui as variáveis x e y trocadas, em relação ao *starterMap*, sendo uma lista de listas o *valueMap* para obter o valor pretendido, o y indica a linha e o x a coluna no *ValueMap* em relação ao *starterMap*.

Bugs e outros problemas encontrados

Neste projeto enfrentamos vários bugs e problemas inesperados em relação ao agente e ao próprio *maze*, de todos os problemas encontrados apenas dois ainda se encontram presentes.

Sendo estes, quando um agente inicia o *Maze* e não há nenhuma saída possível, ou seja, quando o agente fica “bloqueado” com as paredes (blocos cinzentos) impossibilitando a sua saída da célula inicial. Também relacionado com as paredes, ocasionalmente, as chaves encontravam-se em uma das quatro extremidades do *Maze* resultando com que o agente ao apanhar a chave entrasse num loop infinito, como forma de atenuar este erro decidimos colocar as chaves a uma distância relativa das bordas das paredes.

Outro Bug presente é quando o agente não têm nenhum movimento possível, ficando assim preso, tendo em conta que este não pode efetuar o movimento contrário ao anterior, fazendo assim com que em alguns casos, não haja movimentos possíveis. Para contabilizar que o agente já passou por todas as chaves, a chave é removida não estando mais presente no *valueMap*, a cada movimento o *valueMap* é atualizado de forma que o agente não realize movimentos prescindíveis. O programa acaba quando não há chaves, ou seja, a lista das chaves está vazia.

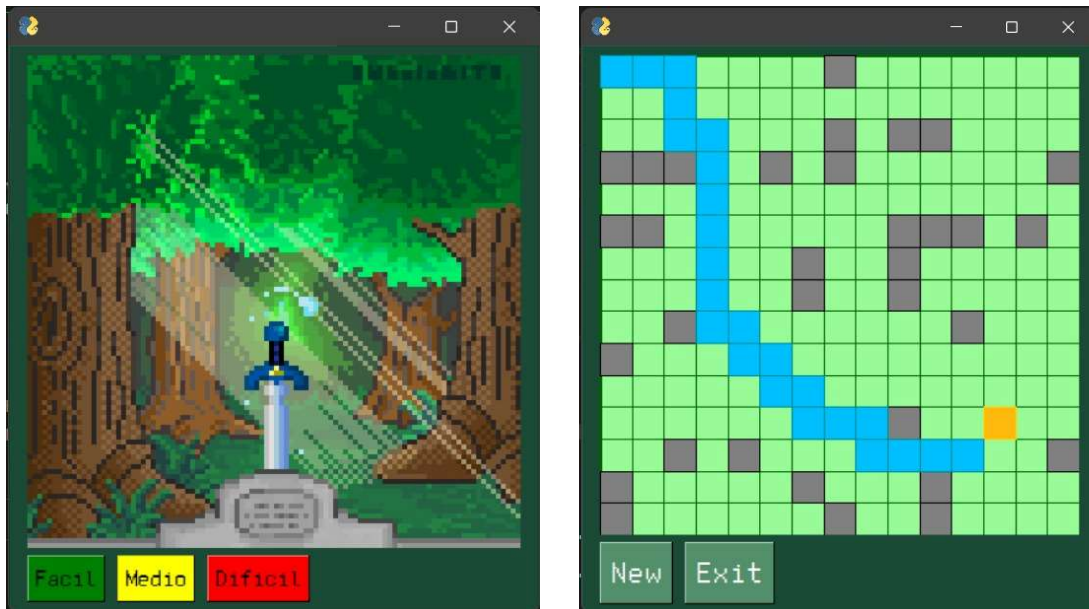


Como esse tipo de bugs dificilmente é resolvido, decidimos colocar no nosso menu, ao lado da opção ‘Exit’ que, anteriormente, era a única forma que tínhamos para sair do programa caso acontecesse um destes bugs, a opção ‘New’ que, achamos uma forma mais viável e apresentável de, ao ocorrer um bug, reiniciava o *maze* e o agente, criando aleatoriamente um novo, sendo muito mais prático e muito mais apresentável para quem o está a ver, evitando assim ter de voltar a dar *run* no programa.

Testes & Resultados

Na parte da testagem de valores de forma a obtermos um *maze* mais eficiente, chegamos aos valores onde o agente, exceto no caso de existência de um dos bugs anteriormente referidos, consegue alcançar a sua finalidade. No código existe um dicionário onde estão presentes os dados importantes para o bom funcionamento do mesmo. O número de linhas é caracterizado pelo *cellCount*, ou seja, o número de células que há na interface e no *maze* por linha. O *gridSize* representa o tamanho da janela da interface, o *rowColCount* indica o número de paredes sendo uma parede uma linha, havendo falhas nessa parede, a variável “falhas” no código retrata o

número de falhas. A quantidade de chaves é representada pela variável $nKeys$ e a variável XY_{real} indica a posição real do agente. Com o objetivo de o agente se mover mais rápido definimos um tempo por movimento sendo esta a variável $SpeedMs$.



Com o intuito de tornar este projeto mais pormenorizado do que o inicialmente feito, decidimos acrescentar no “Menu Inicial” as opções de diferentes dificuldades, onde cada uma tem um percurso mais difícil de executar para o agente e aumentando também progressivamente o número de keys, tornando assim este projeto mais desafiante e colocando ainda mais à prova, o nível de performance do nosso agente.

Conclusão

Uma vez que o projeto está finalizado, vamos agora de forma sucinta sumarizar, as partes mais relevantes do mesmo. Portanto, sendo assim, o nosso projeto baseou-se na criação de um labirinto com o intuito de que um agente consiga, sem qualquer tipo de ajuda, obter as “chaves” necessárias de forma a concluir o labirinto. Apesar de este ter sido um projeto desafiante, penso que conseguimos implementar de forma correta o conceito de *Machine learning*, contudo houve alguns bugs como já referimos anteriormente que não conseguimos resolver, mas sim apenas acrescentar algo que o reinicia de forma autónoma e muito mais apresentável, e que não impedem de forma alguma o funcionamento do programa na grande maioria das situações.

Referências

[30/03/2021] <https://github.com/KenoLeon/Medium-Mazes>

[10/02/2020] https://github.com/ankonzoid/LearningX/tree/master/classical_RL/gridworld

[04/05/2020] <https://www.geeksforgeeks.org/epsilon-greedy-algorithm-in-reinforcement-learning/>

[20/12/2018] <https://towardsdatascience.com/reinforcement-learning-rl-101-with-python-e1aa0d37d43b>

[29/04/----] <https://k3no.medium.com/build-a-maze-with-python-920ac2266fe7>